



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto final

Simulación de Procesos Distribuidos usando Docker, gRPC y
Relojes de Lamport.

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

10 de Junio del 2026

Índice

1	Objetivo	1
2	Planteamiento del escenario:	1
3	Diagrama del escenario:	2
4	Documentacion de codigo:	3
4.1	Codigo proto:	3
4.2	Codigos python:	3
4.3	Codigos docker:	3
5	Secuencia de comandos para la ejecucion:	3
6	Resultados:	3
6.1	Capturas de los resultados:	3
6.2	Bitacoras:	3
7	Conclusion:	3
8	Link de video:	3
	Referencias	3



1. Objetivo

Comprender cómo se mantiene el orden lógico de eventos en sistemas distribuidos mediante el uso de relojes vectoriales.

2. Planteamiento del escenario:

Para el escenario, se tomo base las aplicaciones de reparto de comida donde en este punto tomamos como referencia una arquitectura de microservicios, esto mas precisamente cuando un cliente hace una orden, en nuestro caso tomamos como referencia al restaurante Carls jr, donde tomamos como pedido la hamburguesa Famosa star, esto lo tomamos desde que el usuario selecciona la hamburguesa hasta que el sistema manda la notificación final de entrega, ahora bien, tomamos a los procesos como las responsabilidades.

Para esto, todo comienza en el Proceso 1 que es como tal la Aplicación Cliente, este proceso simula la UI de usuario propiamente, donde como tal el evento interno es agregar la comida al carrito de compras y el monto a pagar, despues de esto, una vez que la orden está lista, el Proceso 1 envía una solicitud de cobro mediante un mensaje punto a punto al Proceso 2 que este sera nuestro movimiento de Pago (donde se verifica si la tarjeta es verdadera y tiene fondos), transmitiendo su reloj vectorial inicial para emepzar la sincronización temporal del sistema.

Despues de esto, al recibir la solicitud, el Proceso 2 actualiza su reloj local y ejecuta un evento interno que es el de validar los datos de la tarjeta y procesar la propia transeaccion, despues tras aprobar el pago, este proceso se comunica con el Proceso 3 que en este punto lo estamos simulando como si fuera la aplicacion que se tiene en la cocina del Restaurante, enviándole la confirmación de fondos aprobados y los detalles de la misma orden, en este caso que tipo de hambrugesa va a desear el cliente, esto lo que hace es que garantiza un orden lógico, donde como tal el restaurante no comenzará a cocinar sin que el pago haya sido exitoso.

Posteriormente, el Proceso 3 registra un evento interno simulando la preparación y el empaquetado de la orden, de hecho una vez que el paquete está listo en el mostrador, la cocina enviara una notificación directa al Proceso 4 que es ya directamente la aplicación del Repartidor. Este cuarto proceso recibe la instrucción, sincroniza su vector, y lanza su propio evento interno para calcular la ruta óptima de entrega mediante el GPS hacia el domicilio del cliente. Al iniciar el viaje, el repartidor actualiza el estado de la orden enviando un mensaje al último proceso.

Ya por ultimo, tenemos al Proceso 5 que es el servidor de notificaciones, donde en este al recibir el mensaje del repartidor, este ejecuta un evento interno para registrar el Tiempo Estimado de Llegada que es el ETA, y para acabar el ciclo de la orden, el Proceso 5 emite un Broadcast hacia los cuatro procesos restantes P1, P2, P3 y P4 indicando que la orden ha sido completada y entregada de forma exitosa.



Este ultimo paso es importante, debido a que es el punto donde el sistema logra sincronizar completamente todos los estados haciendo que los cinco relojes vectoriales sean los mismos de manera simultánea.

Servicios gRPC

1. **Enviar Mensaje:** Recibe una estructura ReqEnvio con los identificadores de origen, destino y de mensaje, esto de forma estructurada como una cadena, despues de esto se incrementa la posición en el Reloj Vectorial del emisor y coloca la línea de envío en la bitácora.
2. **Recibir Mensaje:** Recibe una estructura ReqRecepcion que contiene el proceso de origen, el mensaje y el arreglo del vector, despues de esto actualiza el reloj local calculando los valores máximos, esto por de forma que lo hace componente a componente, e incrementando en 1, por ultimo registra el evento de recepción y devuelve un mensaje de confirmación con el vector del reloj con el tiempo actual.
3. **Evento Interno:** Recibe una solicitud ReqInterno con la descripción de una tarea (armado de carrito, validación de pago, preparación en cocina o cálculo de ruta GPS), despues este incrementa la posición en su reloj lógico vectorial y pone de forma síncrona la entrada en el archivo de registro.
4. **Broadcast:** Recibe una petición de difusión masiva, est emimso incrementa el reloj vectorial del emisor en 1 y atiende de manera por igual al mismo tiempo el estado actual junto con el mensaje a todos los demás procesos para sincronizar el estado de la orden.

3. Diagrama del escenario:

Descripcion del reloj vectorial:

Diagrama:



4. Documentacion de codigo:

4.1. Codigo proto:

4.2. Codigos python:

4.3. Codigos docker:

5. Secuencia de comandos para la ejecucion:

6. Resultados:

6.1. Capturas de los resultados:

6.2. Bitacoras:

7. Conclusion:

8. Link de video:

Referencias

- Docker. (2026). *Docker Documentation*. Consultado el 28 de mayo de 2026, desde <https://docs.docker.com/>
- gRPC. (2026). *gRPC Documentation*. Consultado el 28 de mayo de 2026, desde <https://grpc.io/docs/>
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>